

RETAILMART V3 • SQL

Filtering & Sorting

WhatsApp Announcement

- FILTERING & SORTING

You built tables, set constraints, filled them with data. Now comes the REAL power -- asking questions. Today you learn to talk to RetailMart V3's 2.7 million rows and extract exactly what you need.

Today's Focus:

- SELECT -- choosing which columns to see
- WHERE -- filtering rows by conditions
- AND, OR, NOT -- combining multiple conditions
- LIKE & ILIKE -- pattern matching
- BETWEEN, IN, IS NULL -- range, list, and missing data
- ORDER BY -- sorting results
- LIMIT & OFFSET -- pagination

All practice uses the real RetailMart V3 dataset!

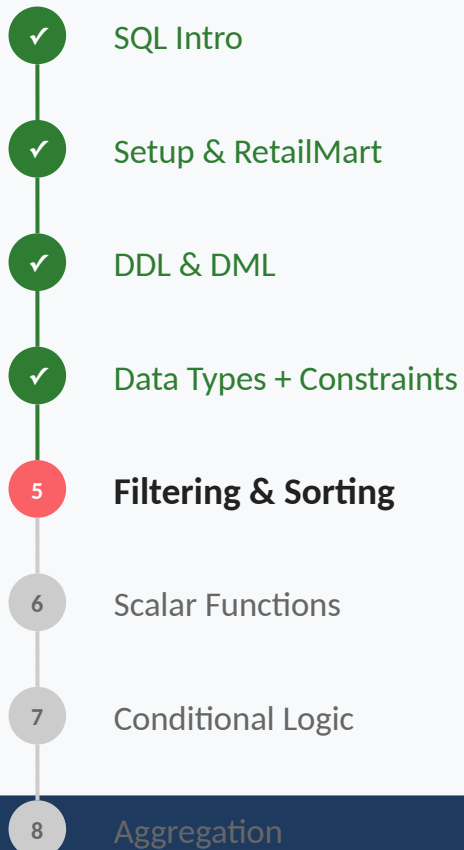
-- Siraj Sir

#Day5 #SELECT #WHERE #PostgreSQL

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: SELECT, WHERE & NULL Handling	2 Hours	SELECT columns FROM table, WHERE with comparison operators (=, !=, >, <, >=, <=), DISTINCT -- removing duplicates, IS NULL / IS NOT NULL, Practice problems (Easy to Hard)
Part 2: Logical Operators & Pattern Matching		AND, OR, NOT -- combining conditions, Operator precedence (AND before OR!), LIKE & ILIKE -- wildcard pattern matching, BETWEEN -- range filtering, IN -- list matching, Practice problems (Easy to Crazy)
Part 3: ORDER BY, LIMIT & OFFSET		ORDER BY ASC/DESC -- sorting results, Multi-column sorting, NULLS FIRST / NULLS LAST, LIMIT & OFFSET -- pagination, SQL query execution order, Practice problems (Easy to Crazy)

YOUR SQL JOURNEY



← HERE

5-Minute Memory Check

5 SQL Categories -- DDL, DML, DQL, DCL, TCL. Today = DQL (SELECT).

RetailMart V3 -- 16 schemas, 55 tables, ~2.7M rows loaded.

DDL -- CREATE / DROP (structure). DML -- INSERT, UPDATE, DELETE, TRUNCATE (data).

Types -- INT, NUMERIC, DATE, BOOLEAN (ALTER ... USING). Constraints -- PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT.

Today's Big Question: RetailMart V3 has 50,000 customers, 150,000 orders, and 1,500 products. How do you find exactly what you need? Answer: SELECT + WHERE.

The Library With 10 Lakh Books

You walk into a HUGE city library with 10 lakh books. You tell the librarian: 'I want to see books.' She gestures at ALL 10 lakh: 'Go ahead.' You would be there for YEARS.

Instead, you narrow it down step by step: 'Only MYSTERY novels' (50,000). 'Published after 2020 AND Indian authors' (2,000). 'Name starts with S' (300). 'Priced 200-500' (80). 'In Hindi OR English' (60). 'Only unique authors, newest first, show me 10.' Done!

From Story to SQL!

'Only MYSTERY novels' -> WHERE genre = 'Mystery'

'After 2020 AND Indian' -> AND year > 2020 AND country = 'India'

'Name starts with S' -> WHERE author LIKE 'S%'

'200 to 500' -> WHERE price BETWEEN 200 AND 500

'Hindi OR English' -> WHERE language IN ('Hindi', 'English')

'Unique authors' -> SELECT DISTINCT author

'Newest first, 10 only' -> ORDER BY year DESC LIMIT 10

The Google Search Analogy

When you type 'best restaurants Bangalore below 500' into Google, you're telling it: WHAT (restaurants), WHERE (Bangalore), CONDITION (below 500). SQL works exactly the same -- but with PRECISE results instead of fuzzy matches.

SELECT -- Data Query Language

Technical:

SELECT retrieves data from one or more tables without modifying anything. It supports column selection, filtering (WHERE), sorting (ORDER BY), and limiting (LIMIT). SELECT is the most used SQL command -- every analyst runs it hundreds of times daily.

Layman's:

SELECT is like asking a librarian: 'Show me the name and price of all Electronics products.' The librarian finds the right shelf (table), picks the columns you asked for, and shows you the matching rows.

SYNTAX: SELECT & WHERE

-- Select ALL columns

```
SELECT * FROM customers.customers LIMIT 10;
```

-- Select SPECIFIC columns (always preferred)

```
SELECT first_name, last_name, email  
FROM customers.customers LIMIT 10;
```

-- WHERE -- filter rows by condition

```
SELECT first_name, email, tier  
FROM customers.customers  
WHERE tier = 'Gold' LIMIT 10;
```

-- Comparison operators: =, !=, <>, >, <, >=, <=

```
SELECT product_name, price FROM products.products  
WHERE price > 50000;
```

DISTINCT & Column Aliases

```
-- Remove duplicates (stores.stores has a city column)
```

```
SELECT DISTINCT city FROM stores.stores  
ORDER BY city;
```

```
-- Count distinct values
```

```
SELECT count(DISTINCT city) AS unique_cities  
FROM stores.stores;
```

```
-- Column aliases with AS
```

```
SELECT first_name AS "First Name",  
       email AS "Email Address"  
FROM customers.customers LIMIT 5;
```

IS NULL / IS NOT NULL

```
-- Find shipments NOT yet delivered (delivered_date is missing)
```

```
SELECT shipment_id, courier_name, shipped_date  
FROM sales.shipments  
WHERE delivered_date IS NULL LIMIT 10;
```

```
-- Find shipments that HAVE been delivered
```

```
SELECT shipment_id, courier_name, delivered_date  
FROM sales.shipments  
WHERE delivered_date IS NOT NULL LIMIT 10;
```

```
-- WRONG: WHERE delivered_date = NULL -> returns 0 rows!
```

```
-- NULL is not equal to anything, not even itself.
```

```
-- ALWAYS use IS NULL or IS NOT NULL.
```

The #1 NULL Mistake

The Mistake:

WHERE phone = NULL returns ZERO rows. SQL uses three-valued logic: NULL = NULL -> UNKNOWN (not TRUE). WHERE needs TRUE to include a row.

The Fix:

ALWAYS use IS NULL or IS NOT NULL. These are special operators designed for NULL checking. Any comparison with NULL using =, !=, >, < returns UNKNOWN.

Practice 1: Find Gold Tier Customers

EASY

SCENARIO: The Loyalty Program Manager Vivek wants to send a special offer to all Gold tier customers. He needs their names, emails, and phone numbers.

TASK: Write a SELECT on customers.customers for tier = 'Gold'. Show first_name, last_name, email, phone. Also count how many Gold tier customers exist.

HINT: Use WHERE tier = 'Gold'. For count, use SELECT count(*) in a separate query.

Answer

✓ ANSWER

```
SELECT first_name, last_name, email, phone  
FROM customers.customers  
WHERE tier = 'Gold';
```

```
SELECT count(*) AS gold_customers  
FROM customers.customers  
WHERE tier = 'Gold';
```

Practice 2: Price Range Investigation

MEDIUM

SCENARIO: The CFO Anita suspects some products are priced too low (below 100 -- possible loss-leaders) and wants to also know premium products (above 50,000). She needs product names and prices for both lists.

TASK: Write FOUR queries on products.products -- (1) price < 100, (2) price > 50000, (3) the healthy middle using BETWEEN 100 AND 50000, (4) BOTH extremes in ONE query using NOT BETWEEN. Count each.

HINT: BETWEEN is inclusive on both ends. NOT BETWEEN flips it -- one query catches both tails at once.

Answer (1/2)

✓ ANSWER

-- Budget products (possible loss-leaders)

```
SELECT product_name, price
FROM products.products
WHERE price < 100
ORDER BY price;
```

-- Premium products

```
SELECT product_name, price
FROM products.products
WHERE price > 50000
ORDER BY price DESC;
```

-- The healthy middle (inclusive both ends)

```
SELECT count(*) AS mid_range
FROM products.products
WHERE price BETWEEN 100 AND 50000;
```

-- BOTH extremes in one query

```
SELECT product_name, price
FROM products.products
WHERE price NOT BETWEEN 100 AND 50000
ORDER BY price;
```

Answer (2/2)

 ANSWER

```
SELECT count(*) FROM products.products WHERE price < 100;  
SELECT count(*) FROM products.products WHERE price > 50000;
```

Practice 3: NULL Data Audit

HARD

SCENARIO: Data Quality lead Rohit is auditing RetailMart V3 for missing data. He needs to know: how many support tickets were never resolved (NULL resolved_date)? How many shipments were never delivered (NULL delivered_date)? How many page views came from anonymous visitors (NULL customer_id)?

TASK: Write 3 count queries using IS NULL. Then prove the count(*) vs count(column) trick on shipments. Which table has the most missing data?

HINT: Use WHERE col IS NULL. count(column) automatically skips NULLs -- so count(*) minus count(delivered_date) = the missing ones.

Answer



```
-- Tickets never resolved
SELECT count(*) FROM support.tickets
WHERE resolved_date IS NULL;    -- 5,502

-- Shipments never delivered
SELECT count(*) FROM sales.shipments
WHERE delivered_date IS NULL;   -- 14,982

-- Anonymous page views (no logged-in customer)
SELECT count(*) FROM web_events.page_views
WHERE customer_id IS NULL;      -- 150,316 <- the biggest gap

-- The count(*) vs count(col) trick
SELECT count(*) AS all_shipments,
       count(delivered_date) AS delivered
FROM sales.shipments;

-- count(col) skips NULLs automatically!
```

Practice 4: Cross-Schema Health Check

CRAZY

SCENARIO: The CTO wants a quick data snapshot across ALL major schemas. Write separate count queries for each table to understand the data scale.

TASK: Write 6 count queries -- customers (total), customers with Gold tier, orders in 2025, products with price > 10000, employees who joined after 2024-06-01, open support tickets.

HINT: Each schema has its own prefix: customers.customers, sales.orders, products.products, stores.employees, support.tickets.

Answer (1/2)

✓ ANSWER

-- Total customers

```
SELECT count(*) AS total FROM customers.customers;
```

-- Gold tier customers

```
SELECT count(*) AS gold  
FROM customers.customers WHERE tier = 'Gold';
```

-- Orders in 2025

```
SELECT count(*) AS orders_2025  
FROM sales.orders  
WHERE order_date >= '2025-01-01';
```

-- Premium products

```
SELECT count(*) AS premium  
FROM products.products WHERE price > 10000;
```

-- Recent hires

```
SELECT count(*) FROM stores.employees  
WHERE joining_date > '2024-06-01';
```

-- Open tickets

```
SELECT count(*) FROM support.tickets
```

Answer (2/2)

✓ ANSWER

```
WHERE status = 'Open';
```

SELECT & WHERE

SELECT reads data without changing anything -- the safest SQL command. WHERE filters rows by condition. Use specific column names (not *). Use IS NULL for NULL checks (never = NULL). DISTINCT removes duplicates. Always add LIMIT on large tables to avoid loading millions of rows.

SELECT * vs Named Columns

Q: 'Why should you avoid SELECT * in production code?'

A: SELECT * is slower (transfers all columns), breaks when schema changes (new columns appear), and exposes sensitive data (passwords, tokens). Always name your columns explicitly. SELECT * is fine for quick exploration, never for production queries or application code.

The Flipkart Filter Analogy

When you shop on Flipkart, you stack filters: Category: Smartphones. Brand: Samsung OR Apple. Price: 15K-40K. Rating: 4+ stars. Each filter NARROWS the results.

In SQL: AND narrows (both must be true). OR widens (either can be true). NOT reverses a condition. These are the building blocks of every complex query.

AND, OR, NOT

Technical:

AND requires ALL conditions to be true. OR requires at least ONE condition to be true. NOT negates a condition. AND has higher precedence than OR (evaluated first), just like multiplication before addition. Use parentheses to control order.

Layman's:

AND = 'I want pizza AND coke' (you need BOTH). OR = 'I want pizza OR burger' (either works). NOT = 'I want anything NOT pizza' (excludes pizza).

SYNTAX: AND, OR, NOT

-- AND: both conditions must be true

```
SELECT * FROM customers.customers  
WHERE tier = 'Gold' AND phone IS NOT NULL;
```

-- OR: either condition can be true

```
SELECT store_name, city FROM stores.stores  
WHERE city = 'Mumbai' OR city = 'Delhi';
```

-- NOT: reverse the condition

```
SELECT * FROM customers.customers  
WHERE tier NOT IN ('Gold', 'Platinum');
```

-- DANGER: AND before OR without parentheses!

-- WRONG: WHERE a = 1 OR b = 2 AND c = 3

-- Reads as: a = 1 OR (b = 2 AND c = 3)

-- FIX: WHERE (a = 1 OR b = 2) AND c = 3

The AND/OR Precedence Trap

The Mistake:

WHERE city = 'Mumbai' OR city = 'Delhi' AND square_ft > 5000 -- looks like 'large stores in Mumbai or Delhi'. But AND runs first! PostgreSQL reads: ALL Mumbai stores + large Delhi stores only.

The Fix:

ALWAYS use parentheses with OR: WHERE (city = 'Mumbai' OR city = 'Delhi') AND square_ft > 5000. Even when not required -- clarity saves debugging hours.

SYNTAX: LIKE, ILIKE, BETWEEN, IN

```
-- LIKE: pattern matching (case-sensitive)
WHERE email LIKE '%@gmail.com'           -- ends with
WHERE product_name LIKE 'Samsung%'       -- starts with
WHERE first_name LIKE 'Ra_ul'            -- _ = 1 char

-- ILIKE: case-insensitive (PostgreSQL-only)
WHERE product_name ILIKE '%samsung%'     -- any case

-- BETWEEN: inclusive range
WHERE price BETWEEN 1000 AND 5000
-- Same as: price >= 1000 AND price <= 5000

-- IN: match against a list
WHERE city IN ('Mumbai', 'Delhi', 'Bangalore')
-- Cleaner than: city = 'Mumbai' OR city = 'Delhi' ...
```

BETWEEN with Timestamps

The Mistake:

WHERE created_date BETWEEN '2025-01-01' AND '2025-01-31' -- if the column is TIMESTAMP, this means up to Jan 31 at 00:00:00. Records on Jan 31 at 1 PM are EXCLUDED!

The Fix:

Use >= and < with the next day: WHERE created_date >= '2025-01-01' AND created_date < '2025-02-01'. This captures ALL of January.

PRO TIP

ILIKE -- PostgreSQL's Secret Weapon

In real apps, users don't care about capitalization. ILIKE matches 'Samsung', 'SAMSUNG', 'samsung' all at once. MySQL doesn't have ILIKE -- you'd need LOWER(column) LIKE LOWER('%text%'). PostgreSQL advantage: cleaner syntax.

Practice 5: Gmail Customers with Gold Tier

EASY

SCENARIO: Email Marketing Manager Pooja wants to target Gmail users who are Gold tier members. She believes they respond better to email campaigns.

TASK: (1) Find customers with email ending '@gmail.com' AND tier = 'Gold'. (2) Pooja types the pattern as '%@GMAIL.COM' -- show that LIKE finds NOTHING but ILIKE still works (case-insensitive). (3) Exclude any test accounts: email NOT ILIKE '%test%'.

HINT: LIKE is case-SENSITIVE ('G' != 'g'). ILIKE ignores case -- the I = Insensitive. NOT ILIKE excludes a pattern.

Answer

✓ ANSWER

```
SELECT first_name, email, tier
FROM customers.customers
WHERE email LIKE '%@gmail.com'
      AND tier = 'Gold'
LIMIT 20;
```

-- LIKE is case-sensitive: uppercase pattern finds NOTHING

```
SELECT count(*) AS with_like
FROM customers.customers
WHERE email LIKE '%@GMAIL.COM';  -- 0 rows!
```

-- ILIKE ignores case: same pattern now matches

```
SELECT count(*) AS with_ilike
FROM customers.customers
WHERE email ILIKE '%@GMAIL.COM';  -- all gmail customers
```

-- Exclude test accounts with NOT ILIKE

```
SELECT count(*) AS gmail_gold_real
FROM customers.customers
WHERE email ILIKE '%@gmail.com'
      AND tier = 'Gold'
      AND email NOT ILIKE '%test%';
```

Practice 6: Order Date Range with Status Filter

MEDIUM

SCENARIO: The ops team needs all orders from January 2025 that are NOT yet delivered. They want to see how many orders are stuck.

TASK: (1) Jan 2025 orders with `order_status != 'Delivered'`. (2) Better business logic: 'stuck' really means NOT IN ('Delivered', 'Cancelled') -- cancelled orders are not stuck. (3) The focus list: status IN ('Processing', 'Out for Delivery'). Compare all three counts.

HINT: Use `>=` and `<` for date ranges. `IN (...)` = match ANY value in the list; NOT IN excludes the whole list. Notice how `!=` vs NOT IN vs IN give three different counts.

Answer

✓ ANSWER

```
SELECT order_id, order_date, net_total, order_status
FROM sales.orders
WHERE order_date >= '2025-01-01'
      AND order_date < '2025-02-01'
      AND order_status != 'Delivered'
ORDER BY order_date;
```

```
-- Better: cancelled orders are NOT 'stuck' - exclude both
SELECT count(*) AS truly_stuck
FROM sales.orders
WHERE order_date >= '2025-01-01'
      AND order_date < '2025-02-01'
      AND order_status NOT IN ('Delivered', 'Cancelled');
```

```
-- The focus list: exactly these two in-flight statuses
SELECT count(*) AS in_flight
FROM sales.orders
WHERE order_date >= '2025-01-01'
      AND order_date < '2025-02-01'
      AND order_status IN ('Processing', 'Out for Delivery');
```

Practice 7: The AND/OR Precedence Trap -- Live Demo

HARD

SCENARIO: A junior developer writes a query to find large stores (`square_ft > 3000`) in Prayagraj or Noida. But the results include SMALL Prayagraj stores too. The bug is operator precedence.

TASK: Show the WRONG query on `stores.stores` and explain why it fails. Fix it with parentheses. Then invert it with NOT: all large stores OUTSIDE those two cities. Compare all three result counts (11 vs 5 vs 84!).

HINT: AND has higher precedence than OR. Without parentheses, '`Prayagraj OR Noida AND square_ft > 3000`' means 'ALL Prayagraj stores + large Noida stores only'. NOT (...) negates a whole bracketed condition.

Answer (1/2)

✓ ANSWER

-- WRONG: Returns ALL 8 Prayagraj stores (any size!) + 3 large Noida = 11 rows

```
SELECT store_name, city, square_ft
FROM stores.stores
WHERE city = 'Prayagraj'
       OR city = 'Noida'
       AND square_ft > 3000;
```

-- CORRECT: Parentheses fix the logic -> only the 5 truly large stores

```
SELECT store_name, city, square_ft
FROM stores.stores
WHERE (city = 'Prayagraj'
       OR city = 'Noida')
       AND square_ft > 3000
ORDER BY square_ft DESC;
```

-- INVERTED with NOT: large stores OUTSIDE the two cities (84 rows)

```
SELECT store_name, city, square_ft
FROM stores.stores
WHERE NOT (city = 'Prayagraj' OR city = 'Noida')
       AND square_ft > 3000
ORDER BY square_ft DESC;
```

Answer (2/2)



-- 11 vs 5 vs 84 - precedence changes EVERYTHING.

Practice 8: Multi-Filter Investigation

CRAZY

SCENARIO: The analytics team needs three separate filtered views of RetailMart data using different filter combinations.

TASK: (1) Find high-value orders (`net_total > 50000`) from 2025, sorted by `net_total` DESC. (2) Find 'Processing' orders older than 30 days -- genuinely stuck. (3) Find customers whose `first_name` starts with 'A' or 'S' and have a phone number.

HINT: Each query targets a single table. Use `LIKE 'A%'` for name patterns. Use `NOW() - INTERVAL '30 days'` for date math.

Answer

✓ ANSWER

-- 1. High-value 2025 orders

```
SELECT order_id, order_date, net_total, order_status
FROM sales.orders
WHERE net_total > 50000
      AND order_date >= '2025-01-01'
ORDER BY net_total DESC LIMIT 20;
```

-- 2. Old 'Processing' orders - genuinely stuck

```
SELECT order_id, order_date, net_total
FROM sales.orders
WHERE order_status = 'Processing'
      AND order_date < NOW() - INTERVAL '30 days'
LIMIT 20;
```

-- 3. Name pattern + phone filter

```
SELECT first_name, last_name, email, phone
FROM customers.customers
WHERE (first_name LIKE 'A%' OR first_name LIKE 'S%')
      AND phone IS NOT NULL
LIMIT 20;
```

Logical Operators & Patterns

AND narrows, OR widens, NOT reverses. ALWAYS use parentheses with OR. LIKE for pattern matching (% = any chars, _ = one char). ILIKE for case-insensitive. BETWEEN for inclusive ranges (watch timestamps!). IN for lists (cleaner than multiple ORs).

LIKE vs ILIKE Performance

Q: 'Is ILIKE slower than LIKE?'

A: Yes, slightly. ILIKE cannot use regular B-tree indexes efficiently. For large tables, create a functional index: `CREATE INDEX idx_lower_name ON products(LOWER(product_name))`. Then use `WHERE LOWER(product_name) LIKE '%samsung%'` instead. Or use `pg_trgm` extension for fast pattern matching.

The Amazon Search Results

When you search 'headphones' on Amazon, you see 10,000 results. But you only see 20 at a time, sorted by relevance. You click 'Price: Low to High' and the sort changes. You click page 2 to see the next 20.

ORDER BY = choosing the sort order. LIMIT = how many results per page. OFFSET = which page you are on. These three commands power every paginated list you have ever seen.

ORDER BY

Technical:

ORDER BY sorts the result set by one or more columns. ASC (ascending) is the default -- smallest first. DESC (descending) = largest first. Multi-column sorting uses left-to-right priority. NULLS FIRST/LAST controls where NULL values appear.

Layman's:

ORDER BY is like telling a librarian: 'Sort these books by price, cheapest first.' ASC = A to Z, 1 to 100, oldest to newest. DESC = Z to A, 100 to 1, newest to oldest.

SYNTAX: ORDER BY

```
-- Sort ascending (default)
SELECT product_name, price FROM products.products
ORDER BY price ASC;

-- Sort descending (most expensive first)
SELECT product_name, price FROM products.products
ORDER BY price DESC;

-- Multi-column sort
SELECT first_name, last_name, registration_date
FROM customers.customers
ORDER BY last_name ASC, registration_date DESC;
-- Sort by last_name A-Z, then newest first within same name

-- NULLS FIRST / NULLS LAST
SELECT first_name, phone FROM customers.customers
ORDER BY phone NULLS LAST;
-- NULLs go to the bottom instead of top
```

LIMIT & OFFSET

Technical:

LIMIT restricts the number of rows returned. OFFSET skips the first N rows. Together they enable pagination: LIMIT 20 OFFSET 40 = page 3 (skip first 40, show next 20). OFFSET without ORDER BY gives unpredictable results -- always sort first.

Layman's:

LIMIT = 'Only show me 10 results.' OFFSET = 'Skip the first 20.' Page 1 = LIMIT 20 OFFSET 0. Page 2 = LIMIT 20 OFFSET 20. Page 3 = LIMIT 20 OFFSET 40.

SYNTAX: LIMIT & OFFSET

```
-- First 10 results
SELECT product_name, price FROM products.products
ORDER BY price DESC LIMIT 10;
```

```
-- Pagination: page_size = 20
-- Page 1:
SELECT * FROM customers.customers
ORDER BY customer_id LIMIT 20 OFFSET 0;
```

```
-- Page 2:
SELECT * FROM customers.customers
ORDER BY customer_id LIMIT 20 OFFSET 20;
```

```
-- Page 3:
SELECT * FROM customers.customers
ORDER BY customer_id LIMIT 20 OFFSET 40;
```

```
-- Formula: OFFSET = (page_number - 1) * page_size
```


OFFSET Performance on Large Tables

OFFSET 1000000 means PostgreSQL scans and discards 1 million rows before returning your results. For deep pagination, use cursor-based pagination instead: `WHERE id > last_seen_id ORDER BY id LIMIT 20`. This is how Twitter, Instagram, and every infinite-scroll app works.

Practice 9: Top 10 Most Expensive Products

EASY

SCENARIO: The CEO's quarterly review is tomorrow. She wants a quick list of the top 10 most expensive products in the RetailMart catalog with their names and prices.

TASK: Query products.products, sort by price DESC, limit to 10. Show product_name, price.

HINT: ORDER BY price DESC puts the most expensive first. LIMIT 10 caps the output.

Answer

✓ ANSWER

```
SELECT product_name, price  
FROM products.products  
ORDER BY price DESC  
LIMIT 10;
```

Practice 10: Paginated Customer List -- Page 3

MEDIUM

SCENARIO: The customer support dashboard shows 20 customers per page. A supervisor needs to see page 3 of all customers sorted alphabetically by last name.

TASK: Return page 3 of customers (20 per page) sorted by last_name ASC. Show customer_id, first_name, last_name, email.

HINT: Page 3 with 20 per page = OFFSET 40 (skip first 2 pages x 20). Formula: $\text{OFFSET} = (\text{page} - 1) \times \text{page_size}$.

Answer

 ANSWER

```
SELECT customer_id, first_name, last_name, email  
FROM customers.customers  
ORDER BY last_name ASC  
LIMIT 20 OFFSET 40;
```

```
-- Page 1: LIMIT 20 OFFSET 0  
-- Page 2: LIMIT 20 OFFSET 20  
-- Page 3: LIMIT 20 OFFSET 40  <- this one  
-- Page N: LIMIT 20 OFFSET (N-1) * 20
```

Practice 11: Multi-Column Sort -- Employee Analysis

HARD

SCENARIO: The HR director needs employees sorted by role alphabetically, and within each role, by salary descending (highest paid first). She only wants employees who joined after 2024-01-01.

TASK: Query stores.employees for joining_date >= '2024-01-01'. Sort by role ASC, then salary DESC. Limit to 30 rows.

HINT: Multi-column ORDER BY uses left column as primary sort. Add WHERE for the date filter.

Answer

 ANSWER

```
SELECT employee_id, first_name, last_name,  
       role, salary, joining_date  
FROM stores.employees  
WHERE joining_date >= '2024-01-01'  
ORDER BY role ASC, salary DESC  
LIMIT 30;
```

Practice 12: RetailMart Data Exploration Report

CRAZY

SCENARIO: You are preparing a data exploration report for the analytics team. They need 5 different sorted views of RetailMart data: most expensive products, newest orders, highest-salary employees, most recent support tickets, and undelivered shipments (oldest first -- the longest-stuck ones).

TASK: Start with a DISTINCT overview -- what order statuses and customer tiers exist at all? Then write 5 sorted views, each with ORDER BY and LIMIT 10, covering products.products, sales.orders, stores.employees, support.tickets, and sales.shipments.

HINT: SELECT DISTINCT col shows each value once -- the fastest way to learn a column's vocabulary. Use DESC for 'most/newest/highest'. Use NULLS LAST for clean output.

Answer (1/2)

✓ ANSWER

-- 0. The vocabulary: DISTINCT values first

```
SELECT DISTINCT order_status FROM sales.orders;
```

```
SELECT DISTINCT tier FROM customers.customers;
```

-- 1. Most expensive products

```
SELECT product_name, price FROM products.products
```

```
ORDER BY price DESC LIMIT 10;
```

-- 2. Newest orders

```
SELECT order_id, order_date, net_total
```

```
FROM sales.orders
```

```
ORDER BY order_date DESC LIMIT 10;
```

-- 3. Highest-salary employees

```
SELECT first_name, last_name, role, salary
```

```
FROM stores.employees
```

```
ORDER BY salary DESC LIMIT 10;
```

-- 4. Most recent support tickets

```
SELECT ticket_id, status, created_date
```

```
FROM support.tickets
```

```
ORDER BY created_date DESC LIMIT 10;
```

Answer (2/2)

✓ ANSWER

```
-- 5. Undelivered shipments, longest-stuck first
SELECT shipment_id, courier_name, shipped_date
FROM sales.shipments
WHERE delivered_date IS NULL
ORDER BY shipped_date NULLS LAST LIMIT 10;
```

ORDER BY, LIMIT & OFFSET

ORDER BY sorts results -- ASC (default) or DESC. Multi-column sort: left column is primary. LIMIT caps rows returned.

OFFSET skips rows for pagination. Formula: $\text{OFFSET} = (\text{page} - 1) \times \text{page_size}$. ALWAYS use ORDER BY with LIMIT -- without it, the order is undefined and results are unpredictable.

Pagination in Production

Q: 'How does pagination work at scale?'

A: LIMIT/OFFSET works for small datasets. For millions of rows, OFFSET becomes slow (scans and discards rows).

Production apps use cursor-based pagination: `WHERE id > 1000 ORDER BY id LIMIT 20`. The client sends the last seen ID, and the server fetches the next page instantly. Twitter, Instagram, and every infinite-scroll feed uses this pattern.

You Write It One Way -- PostgreSQL Reads It Another

You WRITE: SELECT -> FROM -> WHERE -> ORDER BY -> LIMIT. But PostgreSQL EXECUTES in a completely different order. Understanding this order is the key to debugging complex queries.

Execution Order (How PostgreSQL Actually Works)

```
-- You WRITE:                PostgreSQL EXECUTES:
-- 1. SELECT columns          5. SELECT (pick columns)
-- 2. FROM table              1. FROM (identify table)
-- 3. WHERE condition         2. WHERE (filter rows)
-- 4. ORDER BY col           3. ORDER BY (sort)
-- 5. LIMIT n                4. LIMIT (cap output)

-- Example:
SELECT first_name, email      -- Step 5: pick columns
FROM customers.customers     -- Step 1: go to table
WHERE tier = 'Gold'          -- Step 2: filter rows
ORDER BY first_name          -- Step 3: sort result
LIMIT 10;                   -- Step 4: return top 10

-- Why does this matter?
-- You CANNOT use a column alias in WHERE
-- because WHERE runs BEFORE SELECT!
-- SELECT price * 0.9 AS discounted_price
-- WHERE discounted_price > 500; -- ERROR!
-- Fix: WHERE price * 0.9 > 500;
```

Execution Order

FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT. This is why you cannot use aliases in WHERE (runs before SELECT), but CAN use aliases in ORDER BY (runs after SELECT). Understanding this order prevents 90% of 'column not found' errors.

Mistake #1: Single Quotes vs Double Quotes

The Mistake:

WHERE tier = "Gold" -- Double quotes refer to column/table names in PostgreSQL, not string values!

The Fix:

ALWAYS use single quotes for string values: WHERE tier = 'Gold'. Double quotes are for identifiers with special characters: SELECT "First Name" FROM ...

Mistake #2: Forgetting LIMIT on Large Tables

The Mistake:

`SELECT * FROM sales.orders;` -- Returns 150,000+ rows, crashes your pgAdmin, and takes 30 seconds.

The Fix:

Always add LIMIT when exploring: `SELECT * FROM sales.orders LIMIT 10;` Check count first: `SELECT count(*) FROM sales.orders;`

Mistake #3: = NULL Instead of IS NULL

The Mistake:

WHERE phone = NULL returns 0 rows. WHERE phone != NULL also returns 0 rows. Both are wrong.

The Fix:

Use IS NULL and IS NOT NULL. This is the only way to check for NULL in SQL. NULL = NULL -> UNKNOWN, not TRUE.

Mistake #4: OFFSET Without ORDER BY

The Mistake:

`SELECT * FROM customers.customers LIMIT 20 OFFSET 40; --` Without ORDER BY, PostgreSQL returns rows in arbitrary order. Page 2 might overlap with page 1!

The Fix:

ALWAYS use ORDER BY with OFFSET. The sort must be deterministic -- use a unique column (like `customer_id`) as the final sort key.

Mistake #5: BETWEEN With Wrong Order

The Mistake:

WHERE price BETWEEN 5000 AND 1000 -- Returns 0 rows! BETWEEN requires the smaller value FIRST.

The Fix:

Always put the smaller value first: WHERE price BETWEEN 1000 AND 5000. BETWEEN is shorthand for $\geq$ AND $\leq$.

Mistake #6: Case Sensitivity with LIKE

The Mistake:

WHERE product_name LIKE '%samsung%' -- Finds 'samsung' but NOT 'Samsung' or 'SAMSUNG'. LIKE is case-sensitive in PostgreSQL.

The Fix:

Use ILIKE for case-insensitive matching: WHERE product_name ILIKE '%samsung%'. Or use LOWER(): WHERE LOWER(product_name) LIKE '%samsung%'.

Key Tables and Their Columns (1/2)

```
-- Customers (50,000 rows)
-- Columns: customer_id, first_name, last_name,
--          email, phone, registration_date,
--          tier, tier_updated_at
SELECT * FROM customers.customers LIMIT 5;
```

```
-- Products (1,500 rows)
-- Columns: product_id, product_name, brand_id,
--          supplier_id, price, cost_price
SELECT * FROM products.products LIMIT 5;
```

```
-- Orders (150,000 rows)
-- Columns: order_id, cust_id, store_id, order_date,
--          order_status, gross_total, discount_amount,
--          net_total, payment_mode_id
SELECT * FROM sales.orders LIMIT 5;
```

```
-- Stores (200 rows)
-- Columns: store_id, store_name, region_id,
--          city, square_ft, opening_date
```

Key Tables and Their Columns (2/2)

```
SELECT * FROM stores.stores LIMIT 5;
```

More Tables for Practice

```
-- Employees (3,000 rows)
-- Columns: employee_id, store_id, first_name,
--           last_name, email, role, dept_id,
--           joining_date, salary
SELECT * FROM stores.employees LIMIT 5;

-- Support Tickets (20,000 rows)
-- Columns: ticket_id, customer_id, agent_id,
--           category, priority, status,
--           created_date, resolved_date, subject
SELECT * FROM support.tickets LIMIT 5;
```


Explore Before You Query

Before writing a complex query, always explore the table first: (1) `\d schema.table` to see columns and types. (2) `SELECT * FROM table LIMIT 5` to see sample data. (3) `SELECT count(*)` to know the scale. This prevents wrong column names and bad assumptions.

WHERE vs HAVING

Q: 'What is the difference between WHERE and HAVING?'

A: WHERE filters individual rows BEFORE grouping. HAVING filters groups AFTER GROUP BY. WHERE cannot use aggregate functions (count, sum). HAVING can. Example: WHERE tier = 'Gold' filters rows. HAVING count(*) > 5 filters groups with more than 5 members. You will learn GROUP BY and HAVING in detail on Aggregates & Grouping.

LIKE vs Full-Text Search

Q: 'When should you use LIKE vs full-text search?'

A: LIKE is simple pattern matching -- good for exact prefix/suffix searches. Full-text search (tsvector/tsquery in PostgreSQL) is for natural language search with ranking, stemming, and relevance scores. For a search bar, use full-text. For a 'find emails ending with @gmail.com', use LIKE.

NULL in Aggregates

Q: 'How does NULL affect COUNT, SUM, and AVG?'

A: `count(*)` counts ALL rows including NULLs. `count(column)` skips NULLs. SUM and AVG skip NULLs entirely. This means `AVG(salary)` divides only by non-NULL rows -- it does not divide by total rows. Knowing this prevents incorrect calculations in reports.

DISTINCT vs GROUP BY

Q: 'What is the difference between DISTINCT and GROUP BY?'

A: Both can remove duplicates, but GROUP BY is for aggregation (count, sum, avg per group). DISTINCT just removes duplicate rows. `SELECT DISTINCT tier` = unique tiers. `SELECT tier, count(*) GROUP BY tier` = tiers with customer counts. Use DISTINCT for simple deduplication, GROUP BY for analytics.

KEY TAKEAWAYS

1. SELECT reads data without changing it -- the safest SQL command.
2. WHERE filters rows. Supports =, !=, >, <, >=, <= for comparisons.
3. IS NULL / IS NOT NULL -- the ONLY way to check for NULL (never = NULL).
4. DISTINCT removes duplicates. count(DISTINCT col) counts unique values.
5. AND requires all conditions true (narrows). OR requires any (widens). AND runs before OR -- use parentheses!
6. LIKE for patterns (% = any chars, _ = one char). ILIKE for case-insensitive.
7. BETWEEN for inclusive ranges. IN for matching against a list.
8. ORDER BY sorts results -- ASC (default) or DESC. Multi-column sort: left = primary.
9. LIMIT caps output rows. OFFSET skips rows for pagination.
10. Execution order: FROM -> WHERE -> SELECT -> ORDER BY -> LIMIT. You CANNOT use aliases in WHERE.

Filtering & Sorting

-- SELECT

```
SELECT col1, col2 FROM schema.table;  
SELECT * FROM schema.table LIMIT 10;  
SELECT DISTINCT col FROM schema.table;  
SELECT col AS alias FROM schema.table;
```

-- WHERE (Comparison)

```
WHERE col = 'value'  
WHERE col != 'value'  -- or <>  
WHERE col > 100  
WHERE col >= 100  -- or <=
```

-- NULL Handling

```
WHERE col IS NULL  
WHERE col IS NOT NULL  
-- NEVER: WHERE col = NULL (returns 0 rows!)
```

Filtering & Sorting

-- Logical Operators

```
WHERE a = 1 AND b = 2
WHERE a = 1 OR b = 2
WHERE NOT a = 1
WHERE (a = 1 OR b = 2) AND c = 3  -- parentheses!
```

-- Pattern Matching

```
WHERE col LIKE 'abc%'      -- starts with
WHERE col LIKE '%abc'      -- ends with
WHERE col LIKE '%abc%'     -- contains
WHERE col ILIKE '%abc%'    -- case-insensitive
WHERE col LIKE 'A_B'       -- _ = one char
```

-- BETWEEN & IN

```
WHERE col BETWEEN 100 AND 500  -- inclusive
WHERE col IN ('a', 'b', 'c')
WHERE col NOT IN ('x', 'y')
WHERE col NOT BETWEEN 100 AND 500
```


Filtering & Sorting

-- ORDER BY

```
ORDER BY col ASC           -- ascending (default)
ORDER BY col DESC          -- descending
ORDER BY col1 ASC, col2 DESC -- multi-column
ORDER BY col NULLS LAST    -- NULLs at bottom
```

-- LIMIT & OFFSET

```
LIMIT 10                    -- first 10 rows
LIMIT 20 OFFSET 40         -- page 3 (20/page)
-- Formula: OFFSET = (page - 1) * page_size
```

CHALLENGE

Filtering & Sorting Take-Home Challenge: RetailMart Data Detective (1/3)

You are a Data Analyst at RetailMart. Write queries to answer these business questions:

Filtering & Sorting Take-Home Challenge: RetailMart Data Detective (2/3)

- 1.: How many customers have each tier? Write a count query for each tier value: Gold, Silver, Bronze, etc.
- 2.: Find all Gmail customers (email LIKE '%@gmail.com') who are Gold or Silver tier. (LIKE + IN + AND)
- 3.: Find products priced between 5,000 and 20,000, sorted by price DESC. (BETWEEN + ORDER BY)
- 4.: Find the 10 most recent orders above 10,000. (WHERE net_total > 10000 + ORDER BY + LIMIT)
- 5.: Find all support tickets with status 'Open' that have been open more than 7 days. (WHERE + date comparison on created_date)
- 6.: Paginate the employee list: show page 5 with 15 employees per page, sorted by salary DESC. (ORDER BY + LIMIT + OFFSET)
- 7.: Find customers whose first_name starts with 'Pr' and phone IS NOT NULL. (LIKE + IS NOT NULL)
- 8.:

CHALLENGE

Filtering & Sorting Take-Home Challenge: RetailMart Data Detective (3/3)

Find stores in Mumbai or Delhi with square_ft > 3000. Use parentheses correctly with OR + AND.

Push your SQL file to GitHub!

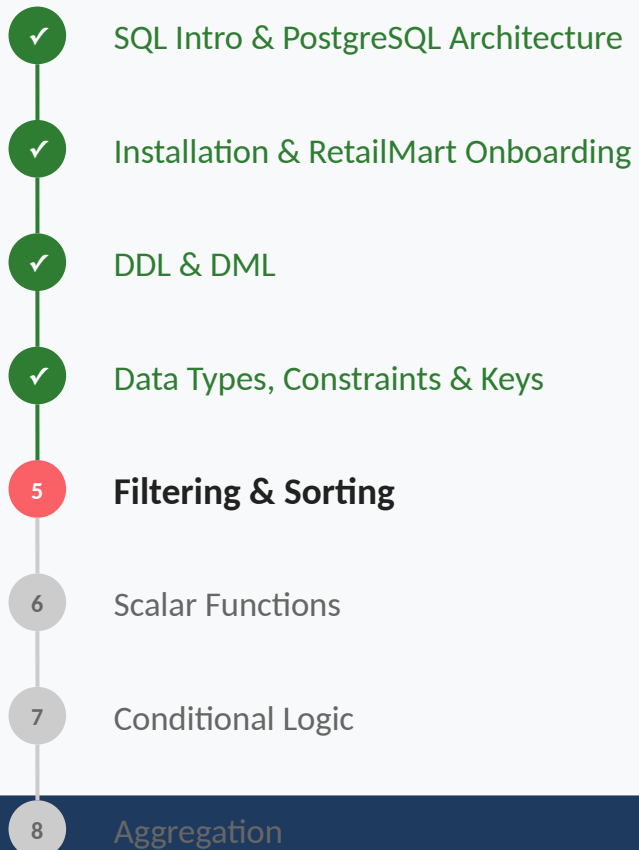
<https://github.com/starlordali4444/postgresql>

WHAT YOU ACHIEVED TODAY

You can now QUERY a real database. SELECT, WHERE, AND, OR, LIKE, BETWEEN, IN, IS NULL, ORDER BY, LIMIT, OFFSET -- these are the tools data analysts use 100+ times every day. You went from building tables to asking questions from them.

Tomorrow: Scalar Functions -- UPPER, LOWER, LENGTH, SUBSTRING, NOW(), EXTRACT, DATE_TRUNC. You will learn to transform and format data.

YOUR SQL JOURNEY



← HERE

Coming Next

earlier taught you to READ data. earlier teaches you to TRANSFORM and ANALYZE it. Scalar functions, conditional logic, aggregation, and joins -- these turn raw rows into business insights.

See you on Scalar Functions!